

Guía de Estudio — Clase 13A

Curso: Laboratorio de Programación y Lenguajes 2026 (IF009) **Institución:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Módulo:** X | **Semana:** 13 | **Clase:** 13A (Clase N° 1 de 1) **Tema oficial:** Subprogramas, Parámetros y Sobrecarga **Duración:** 120 minutos
Lenguaje principal: TypeScript **Bibliografía base:** Sebesta, *Concepts of Programming Languages* (Pearson, 2019), Cap. 11 — Abstract Data Types, Encapsulation, Modules

⚠ Drift detectado — Leé esto antes de empezar

El tema se llama “**Subprogramas, Parámetros y Sobrecarga**” en el plan oficial (`topic.yaml`), pero **el contenido real dado en clase** (registrado en `clase_dada.txt` , la línea base de la clase dada) trata sobre:

- **Tipo Abstracto de Datos (TAD / ADT)**
- **Encapsulamiento vs. ocultamiento de información**
- **Implementación de `Stack<T>` en TypeScript**
- **Interfaz pública y copia defensiva**
- **Independencia de representación**
- **Separación especificación / implementación**
- **Módulo como frontera de visibilidad y compilación separada**
- **Genéricos y restricciones (`T extends Comparable<T>`)**

Esta guía es **fiel al contenido real de la clase** (TAD, módulos, interfaces y genéricos), no al título nominal. El drift está documentado por el Dr. Roberto en `diseño.md` (sección “*Drift detectado vs. título nominal*”). Si ves el título y esperabas subprogramas, parámetros formales y sobrecarga de métodos, no te confundas: **lo que estudies acá es lo que se dictó**. El título se conserva solo por consistencia con el plan de cursada.

📖 **Consejo de Sofía:** Si un alumno puede estudiarlo solo, lo hicimos bien. Esta guía profundiza la clase para que la puedas repasar a tu ritmo, con ejemplos, autoevaluación y glosario. No reemplaza la clase: la extiende.

1. Introducción al tema

Esta clase es la continuación natural de la idea de **abstracción** que se construyó en las clases anteriores. El hilo conductor es claro: parte de un **problema concreto** (el acoplamiento a la representación) y construye, una a una, cuatro herramientas formales que lo resuelven.

Concepto formal	Pregunta que responde
TAD / ADT	¿Qué significa usar un tipo sin ver su representación?
Interfaz	¿Qué operaciones puede asumir el cliente?
Módulo	¿Dónde se declara una frontera de visibilidad?
Genéricos	¿Cómo reutilizar sin mezclar tipos?

Cada concepto responde a una **pregunta de diseño distinta**. No son sinónimos. Si los confundimos, terminamos con clases enormes o con representación expuesta. TypeScript es el lenguaje vehículo porque permite expresar los cuatro conceptos de forma directa: privacidad real con `#`, interfaces estructurales, módulos por archivo y genéricos con restricciones.

Por qué importa este tema: La abstracción de datos es el hito que convirtió a la programación en una disciplina de ingeniería. Sin TADs, cualquier cambio interno rompe código externo; con TADs, el *cómo* puede cambiar sin romper el *quién* depende del contrato. [Sebesta, *Concepts of Programming Languages*, Cap. 11, §11.1, p. 471]

2. Objetivos de aprendizaje

Al finalizar esta guía el estudiante podrá:

1. **Explicar** el problema del acoplamiento a la representación y por qué rompe la abstracción.
 2. **Definir** un Tipo Abstracto de Datos (TAD) por nombre, operaciones, semántica observable e invariantes.
 3. **Clasificar** las operaciones de un TAD en constructoras, transformadoras y observadoras, justificando qué pertenece a la interfaz.
 4. **Distinguir** encapsulamiento, ocultamiento de información, interfaz pública e invariante como cuatro preguntas de diseño distintas.
 5. **Implementar** un TAD pila en TypeScript usando `#` para privacidad real y `T | undefined` para errores definidos.
 6. **Aplicar** la copia defensiva para proteger la representación privada al exponer vistas.
 7. **Justificar** la independencia de representación a partir de un contrato estable.
 8. **Separar** especificación (`interface`) e implementación (`class`) en TypeScript.
 9. **Explicar** el módulo como frontera de visibilidad y los cuatro tipos de imports.
 10. **Usar** genéricos y restricciones (`T extends Comparable<T>`) para reutilizar abstracción sin mezclar tipos.
-

3. Conceptos previos necesarios

Esta guía asume que ya viste los siguientes temas anteriores. No los re-explicamos, pero los usamos como base:

Tema previo	Qué necesitás recordar
Tema 09 — Tipos y sistemas de tipos	Diferencia entre tipo estático y dinámico; tipado estructural vs. nominal.
Tema 10 — Clases y objetos	Sintaxis de <code>class</code> en TypeScript; campos, métodos, <code>this</code> .
Tema 11 — Abstracción	Idea de abstracción como separación entre <i>qué</i> hace algo y <i>cómo</i> lo hace.
Tema 12 — Modularidad 1	Noción de módulo como unidad de organización del código.

Si alguno de estos puntos no te resulta familiar, conviene repasar la guía del tema correspondiente antes de seguir.

4. Desarrollo teórico

Bloque 1 — El problema: acoplamiento a la representación [F-02]

Empecemos por el problema que motiva toda la clase. Imaginate que tenés una pila y el cliente **sabe** que internamente es un array. ¿Qué pasa? El cliente deja de usar una pila y pasa a usar un **array disfrazado**:

```
// Cliente acoplado a un detalle interno:  
// "la pila es un array"  
// El cliente ya no usa una pila  
// Usa un array disfrazado:  
pila.datos[0] = 99  
pila.datos.reverse()
```

Consecuencia: cualquier cambio interno rompe código externo. Si mañana cambiás el array por una lista enlazada, todo el código que accede a `datos` se rompe. La abstracción aparece **recién cuando el cliente solo puede usar operaciones permitidas**. Ese es el punto de partida.

Cita: Sebesta describe exactamente este escenario: *“Suppose that the original implementation of the stack abstraction uses a linked list representation. At a later time, because of memory management problems with that representation, the stack abstraction is changed to use a contiguous [array] representation”* — el cambio solo es seguro si los clientes no dependen de la representación. [Sebesta, *Concepts of Programming Languages*, Cap. 11, §11.2, p. 471]

Idea clave: El acoplamiento a la representación es el síntoma. La abstracción es la cura.

Bloque 2 — TAD: Tipo Abstracto de Datos [F-03, F-04, F-05]

Un **TAD** no se define por *cómo* guarda los datos, sino por cuatro cosas:

TAD = Nombre + Operaciones + Semántica observable + Invariantes

Parte del TAD	Qué ve el cliente	Qué queda oculto
Nombre del tipo	<code>Stack<T></code>	Si usa array, lista o buffer
Operaciones	<code>push</code> , <code>pop</code> , <code>peek</code>	Algoritmos auxiliares
Semántica	LIFO	Cómo se mantiene el orden
Errores definidos	<code>pop()</code> puede no devolver elemento	Caso interno de pila vacía

Esa separación entre lo **visible** y lo **oculto** es el corazón del TAD. No es ocultar por ocultar: es definir qué prometo y qué me reservo cambiar.

Cita: “A user-defined abstract data type should provide the same characteristics as those of language-defined types, such as a floating-point type: (1) a type definition that allows program units to declare variables of the type but hides the representation of the type.” [Sebesta, *Concepts of Programming Languages*, Cap. 11, §11.2.2, p. 471]

La pila como ejemplo mínimo

La pila es una buena explicación de un ADT porque tiene una regla simple: **LIFO** (Last In, First Out). Esa es una **invariante observable**: el array no la garantiza, por eso la pila es un TAD y el array no.

Operación	Promesa observable	Detalle irrelevante
<code>push(x)</code>	Agrega x como próximo candidato a salir	Dónde se almacena
<code>pop()</code>	Devuelve y remueve el último agregado disponible	Cómo se mueve el índice interno
<code>peek()</code>	Observa sin remover	Si usa cache o cálculo directo
<code>isEmpty()</code>	Informa si hay elementos disponibles	Si guarda size o lo calcula

Cada operación separa **promesa observable** de **detalle irrelevante**. Esa es la independencia entre lo que prometo y lo que me reservo.

Cita: Gabbrielli & Martini formalizan esta clasificación: “An ADT without constructors is completely useless. There is no way to construct a value. In general, an ADT must have at least one operation in each of the above categories.” [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, Cap. 9, §9.x, p. 283]

Clasificación de operaciones

Una interfaz **no debería ser una lista arbitraria de métodos**. Una operación pública pertenece al TAD si respeta su semántica.

- `push` / `pop` / `peek` → sí son operaciones de pila.
- `at(index)` → **rompe** la abstracción de pila (convierte la pila en acceso aleatorio).

Tipo de operación	Qué hace	Ejemplo en <code>Stack<T></code>
Constructora	Crea valores del TAD	<code>new Stack<T>()</code> , <code>emptyStack()</code>
Transformadora	Cambia el estado abstracto	<code>push</code> , <code>pop</code> , <code>clear</code>
Observadora	Consulta sin romper la abstracción	<code>peek</code> , <code>isEmpty</code> , <code>size</code>

Cita: Louden & Lambert presentan esta misma clasificación algebraica para el TAD pila: *“the operations createstk and push are constructors, the pop and top [son destructores/transformadores]”* y discuten la especificación algebraica con axiomas. [Louden & Lambert, *Programming Languages: Principles and Practices*, Cap. 11, §11.2, p. 498]

Idea clave: Decidir qué publicar es decidir qué promesas asumir. Publicar de más es asumir promesas de más.

Bloque 3 — Encapsulamiento ≠/ocultamiento de información [F-06]

Esto se confunde siempre. Son **cuatro preguntas de diseño distintas**, no sinónimos:

Concepto	Pregunta que responde	Error común
Encapsulamiento	¿Qué datos y operaciones forman una unidad?	Poner todo en una clase enorme
Ocultamiento de información	¿Qué decisiones internas pueden cambiar?	Exponer campos, arrays o conexiones
Interfaz pública	¿Qué promesas recibe el cliente?	Publicar helpers por comodidad
Invariante	¿Qué debe ser siempre cierto?	Permitir estados imposibles

- **Encapsular agrupa.**
- **Ocultar información protege cambios futuros.**

Confundirlos produce clases enormes (mal encapsulamiento), representación expuesta (mal ocultamiento), helpers públicos que no deberían ser promesa (mala interfaz) o estados imposibles (invariante rota).

Cita: *“Encapsulation and information hiding represent two of the cardinal points of data abstraction. From the linguistic viewpoint, there is not much to add to what we have already said. Every language allows the definition of objects by hiding some part of them (either data or methods).”* [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, Cap. 9, §9.x, p. 295]

Idea clave: Cuatro preguntas, cuatro errores. Si las confundís, el diseño se degrada.

Bloque 4 — Implementación de Stack en TypeScript [F-07]

TypeScript permite expresar el TAD con una clase genérica. El cliente usa operaciones; no toca la estructura interna.

```
class Stack<T> {
  #items: T[] = []
  //# propiedad privada

  push(item: T): void {
    this.#items.push(item)
  }

  pop(): T | undefined {
    return this.#items.pop()
  }

  peek(): T | undefined {
    return this.#items.at(-1)
  }

  isEmpty(): boolean {
    return this.#items.length === 0
  }
}
```

Detalles clave del código:

- `#items` es **privacidad real** en TypeScript (ES2022): el campo no es accesible desde fuera ni siquiera en runtime. No es lo mismo que `private`, que solo afecta el tipado.
- `pop()` devuelve `T | undefined` porque la pila puede estar vacía: ese `undefined` es el **error definido** del TAD. Lanzar una excepción sería otra decisión de diseño válida, pero distinta.
- `peek()` usa `this.#items.at(-1)` para mirar el tope sin remover.
- `isEmpty()` consulta `length`. Todo respeta la semántica LIFO.

Cita: Sebesta presenta un ejemplo análogo en C++: una clase `Stack` con miembros `private` (`stackPtr`, `maxLen`, `topSub`) y miembros `public` (las operaciones). La estructura conceptual es la misma: representación oculta, operaciones expuestas. [Sebesta, *Concepts of Programming Languages*, Cap. 11, §11.4.1.4, p. 471]

Idea clave: Esta clase es un TAD pila: nombre, operaciones, semántica e invariantes, todo en una sola unidad.

Bloque 5 — ¿Qué debe exponer una interfaz? [F-08]

La interfaz debe respetar la **semántica del TAD**, no ser la lista de todo lo que se puede hacer con la clase.

Operación	¿Conviene?	Motivo
<code>clear()</code>	Sí	Puede ser operación válida de pila
<code>toArray()</code>	Depende	Debe devolver copia o vista de solo lectura
<code>at(index)</code>	No	Convierte la pila en acceso aleatorio

```
export interface Stack<T> {
  push(item: T): void
  pop(): T | undefined
  peek(): T | undefined
  isEmpty(): boolean
  clear(): void
  toArray(): readonly T[]
}
```

Si una operación **revela la representación**, deja de ser abstracción. La interfaz pública es un contrato: cada método que publicamos es una promesa que el cliente asume.

Cita: “The description of the semantics of the operations of an ADT is a specification, expressed not in terms of concrete types but general abstract relations.” La interfaz es esa especificación: declara las relaciones abstractas, no los detalles concretos. [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, Cap. 9, §9.x, p. 283]

Idea clave: Publicar de más = asumir promesas de más. La interfaz no es una lista de capacidades: es la lista de lo que respeta la semántica del TAD.

Bloque 6 — Copia defensiva [F-09]

El patrón clave para proteger la representación cuando hay que exponer una vista.

Mal diseño — expone la representación interna:

```
class Stack<T> {
  #items: T[] = []

  toArray(): T[] {
    return this.#items // expone representación interna
  }
}
```

Si el cliente hace `pila.toArray().push(99)`, modifica la pila **sin pasar por** `push`. Eso rompe todo.

Mejor diseño — copia superficial:

```
class Stack<T> {
  #items: T[] = []

  toArray(): readonly T[] {
    return [...this.#items] // copia superficial
  }
}
```

El cliente recibe una vista nueva; si la muta, no afecta a la pila. `readonly T[]` refuerza la inmutabilidad de la vista en el tipo.

Cita: Sebesta vincula esto directamente con la ventaja del ocultamiento de información: el cambio de representación (de lista enlazada a array contiguo) solo es seguro si los clientes no pueden mutar la estructura interna directamente. [Sebesta, *Concepts of Programming Languages*, Cap. 11, §11.2, p. 471]

Idea clave: Cualquier método que devuelva la estructura interna debe devolver una copia o una vista de solo lectura. La copia defensiva protege la representación privada.

Nota sobre copia superficial: Para tipos primitivos (`number` , `string` , `boolean`), la copia superficial alcanza. Para objetos mutables internos, no: habría que copiar profundo o devolver vistas inmutables recursivas. Hay un costo de performance, pero es el precio del encapsulamiento. Si la pila es enorme, se puede devolver un iterador de solo lectura.

Bloque 7 — Independencia de representación [F-10]

Dos implementaciones pueden tener el **mismo contrato**.

Implementación	Representación interna	Cliente que usa <code>Stack<T></code>
<code>ArrayStack<T></code>	Array <code>T[]</code>	No cambia
<code>LinkedStack<T></code>	Nodos enlazados	No cambia
<code>BoundedStack<T></code>	Array con capacidad máxima	Cambia solo si cambia el contrato
<code>PersistentStack<T></code>	Estructura inmutable compartida	No cambia si conserva LIFO

El cliente **no debe saber** si la pila usa array, nodos o buffer. Cambiar la representación interna no rompe a quien depende del contrato.

Independencia de representación = poder cambiar el *cómo* sin romper el *quién*.

Cita: “Abstract data types guarantee the encapsulation and hiding of information but they are rigid when used in a design with a degree of complexity.” La solución a esa rigidez son los módulos, que permiten múltiples implementaciones detrás de un mismo contrato. [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, Cap. 9, §10.1.1, p. 295]

Idea clave: Mismo contrato → múltiples implementaciones intercambiables. El cliente cambia solo si cambia el contrato, no si cambia la representación.

Bloque 8 — Separación entre especificación e implementación [F-11]

Para que la independencia de representación sea efectiva, necesitamos separar **sintácticamente** el contrato del cuerpo.

```
export interface Stack<T> {
  push(item: T): void
  pop(): T | undefined
  peek(): T | undefined
  readonly size: number
}

export class ArrayStack<T> implements Stack<T> {
  #items: T[] = []

  push(item: T): void {
    this.#items.push(item)
  }

  pop(): T | undefined {
    return this.#items.pop()
  }

  peek(): T | undefined {
    return this.#items.at(-1)
  }

  get size(): number {
    return this.#items.length
  }
}
```

- La interfaz dice qué promete.
- La implementación decide cómo se cumple.

Si mañana escribís `LinkedList<T>`, implementás la misma interfaz con otra representación. El cliente que depende de `Stack<T>` no se entera. Esa es la diferencia entre **depender de un contrato** y **depender de una implementación**.

Cita: Louden & Lambert discuten los mecanismos de TAD y módulos precisamente en términos de modificabilidad, reusabilidad y seguridad: la separación especificación/implementación habilita las tres. [Louden & Lambert, *Programming Languages: Principles and Practices*, Cap. 11, §11.2, p. 496]

Idea clave: `readonly size: number` en la interfaz → observable sin mutación. `get size(): number` en la clase → calcula sin exponer el campo. La separación sintáctica habilita la independencia de representación.

Bloque 9 — Módulo como frontera de visibilidad [F-12]

Un **módulo** decide qué nombres salen al exterior. En TypeScript, cada archivo es un módulo.

```
// stack.ts
export type { Stack }
export { ArrayStack }

class NodeStack<T> implements Stack<T> {
  // Implementación alternativa interna.
}

function validarCapacidad(max: number): void {
  // Helper privado del módulo.
}
```

- `exportado` → parte de la **frontera pública**.
- `no exportado` → **detalle interno** del módulo.

TypeScript separa contrato y cuerpo con `interface` y `class`:

```
export interface IStack<T> {
  push(item: T): void;
  pop(): T | undefined;
  peek(): T | undefined;
  readonly size: number;
}

export class ArrayStack<T> implements IStack<T> {
  private readonly datos: T[] = [];
  push(item: T): void { this.datos.push(item); }
  pop(): T | undefined { return this.datos.pop(); }
  peek(): T | undefined { return this.datos.at(-1); }
  get size(): number { return this.datos.length; }
}
```

Cita: “A module mechanism can document the dependencies of a module on other modules by requiring explicit import lists whenever code from other modules is used. These dependencies can be used by a compiler to automatically recompile out-of-date modules.” [Louden & Lambert, *Programming Languages: Principles and Practices*, Cap. 11, §11.3, p. 502]

Idea clave: El módulo es la **unidad de visibilidad**, no la clase. Lo que exporto es promesa; lo que no exporto es implementación.

Bloque 10 — Imports, dependencias y compilación separada [F-13]

Compilación separada **no significa compilar a ciegas**. El compilador necesita contratos.

Import	Tipo de dependencia	Pregunta
<code>import type { Stack }</code>	Dependencia estática de tipos	¿Alcanza con el contrato?
<code>import { ArrayStack }</code>	Dependencia de implementación	¿Es correcto acoplarme a ella?
<code>import { compare } from "./order.js"</code>	Dependencia funcional	¿Pertenece a este módulo?
<code>import "./polyfill"</code>	Dependencia por efecto lateral	¿Está documentada?

Cada import es una **decisión de acoplamiento**. El código correcto depende solo del contrato:

```
import type { Stack } from "./stack.js"
export function mover<T>(origen: Stack<T>, destino: Stack<T>): void {
  while (origen.size > 0) {
    const item = origen.pop()
    if (item !== undefined) destino.push(item)
  }
}
```

La función `mover` opera sobre cualquier `Stack<T>` sin saber si es `ArrayStack` o `LinkedStack`. `import type` se borra en runtime: es solo para chequeo de tipos. Esa es la compilación separada bien entendida.

Cita: Gabbrielli & Martini describen la cláusula de imports como parte de la definición de módulo: un módulo puede usar definiciones *“of another module by importing it (the imports clause)”*. [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, Cap. 9, §9.x, p. 283]

Idea clave: `import type` → dependencia solo del contrato (lo más débil). Importar implementación sin necesitarla te acopla a detalles que pueden cambiar.

Bloque 11 — Genéricos [F-14]

Sin genéricos: `NumberStack`, `StringStack`, `UserStack`. Una clase por cada tipo (duplicación).

Con genéricos: `Stack<T>` parametriza el tipo una sola vez.

```
const numeros = new ArrayStack<number>()
numeros.push(10)
numeros.push("hola") // error: string no es number
const n = numeros.pop() // number | undefined
```

El genérico reutiliza la abstracción sin mezclar tipos. Pero hay más: las interfaces pueden ser **restricciones**.

```
interface Comparable<T> {
  compareTo(other: T): number
}

class SortedSet<T extends Comparable<T>> {
  #items: T[] = []
  add(item: T): void {
    this.#items.push(item)
    this.#items.sort((a, b) => a.compareTo(b))
  }
}
```

`Comparable<T>` declara `compareTo(other: T): number`. `SortedSet<T extends Comparable<T>>` exige que `T` sea comparable. El genérico no es solo un placeholder: es una **restricción que el compilador hace cumplir**.

Cita: *“Parametric polymorphism is provided by a subprogram that takes generic parameters that are used in type expressions that describe the types of the parameters of the subprogram. Different instantiations of such subprograms can be given different generic parameters, producing subprograms [con tipos distintos].”* [Sebesta, *Concepts of Programming Languages*, Cap. 9, §9.x, p. 389]

Cita: “A value exhibits universal parametric polymorphism when it has an infinite number of different types which can be obtained by instantiating a single schema.” [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, Cap. 8, Def. 8.6, p. 136]

Idea clave: El genérico es **polimorfismo paramétrico**: mismo cuerpo, tipos distintos. No confundir con **sobrecarga** (ad hoc polymorphism: mismo nombre, cuerpos distintos). `T extends Comparable<T>` es una restricción recursiva: el tipo se refiere a sí mismo, común en patrones como `Comparable` o `Cloneable`.

5. Ejemplos trabajados

Ejemplo 1 — Implementar `Stack<T>` con `ArrayStack<T> implements IStack<T>`

Consigna: Implementar un TAD pila en TypeScript que separe especificación (`interface`) de implementación (`class`), con privacidad real y error definido.

Paso 1 — Definir el contrato (especificación).

```
export interface IStack<T> {  
  push(item: T): void  
  pop(): T | undefined  
  peek(): T | undefined  
  isEmpty(): boolean  
  readonly size: number  
}
```

La interfaz declara las operaciones que respeta la semántica LIFO. `size` es `readonly`: observable sin mutación. `pop` devuelve `T | undefined`: el error definido (pila vacía).

Paso 2 — Implementar con representación oculta.

```
export class ArrayStack<T> implements IStack<T> {  
  #items: T[] = []  
  
  push(item: T): void {  
    this.#items.push(item)  
  }  
  
  pop(): T | undefined {  
    return this.#items.pop()  
  }  
  
  peek(): T | undefined {  
    return this.#items.at(-1)  
  }  
  
  isEmpty(): boolean {  
    return this.#items.length === 0  
  }  
  
  get size(): number {  
    return this.#items.length  
  }  
}
```

Paso 3 — Verificar que el cliente depende del contrato, no de la implementación.

```
import type { IStack } from "./stack.js"
import { ArrayStack } from "./stack.js"

const pila: IStack<number> = new ArrayStack<number>()
pila.push(10)
pila.push(20)
console.log(pila.peek()) // 20
console.log(pila.pop()) // 20
console.log(pila.size) // 1
```

Análisis: El cliente declara `pila: IStack<number>` (contrato) y construye con `new ArrayStack<number>()` (implementación). Si mañana cambiás `ArrayStack` por `LinkedStack`, la línea de declaración no cambia: solo cambia la construcción. Esa es la independencia de representación en acción.

Ejemplo 2 — Análisis de copia defensiva para evitar aliasing

Consigna: Dado el siguiente código, identificar el problema y corregirlo con copia defensiva.

```
class Stack<T> {
  #items: T[] = []

  toArray(): T[] {
    return this.#items
  }
}
```

Paso 1 — Identificar el problema.

`toArray()` devuelve `this.#items` directamente. Aunque `#items` es privado, el array que recibe el cliente es **el mismo objeto** en memoria. El cliente puede mutarlo sin pasar por las operaciones del TAD:

```
const pila = new Stack<number>()
pila.push(1)
pila.push(2)
const vista = pila.toArray()
vista.push(99) // muta el array interno de la pila
vista.reverse() // invierte el orden interno
// La pila ya no respeta LIFO: la invariante se rompió.
```

Esto se llama **aliasing**: dos referencias apuntan al mismo objeto mutable. La abstracción se rompe porque el cliente bypassó las operaciones del TAD.

Paso 2 — Corregir con copia defensiva.

```
class Stack<T> {
  #items: T[] = []

  toArray(): readonly T[] {
    return [...this.#items] // copia superficial
  }
}
```

`[...this.#items]` crea un **array nuevo** con los mismos elementos. Si el cliente muta la vista, no afecta a la pila. `readonly T[]` refuerza en el tipado que la vista es de solo lectura.

Paso 3 — Verificar.

```
const pila = new Stack<number>()
pila.push(1)
pila.push(2)
const vista = pila.toArray()
// vista.push(99) // error de tipado: readonly T[] no tiene push
console.log(pila.size) // sigue siendo 2
```

Análisis: La copia defensiva protege la representación privada. El costo es una copia $O(n)$, pero es el precio del encapsulamiento. Para pilas enormes, se puede devolver un iterador de solo lectura en lugar de un array.

Ejemplo 3 — `SortedSet<T extends Comparable<T>` con constraint genérica

Consigna: Implementar un conjunto ordenado que solo acepte elementos comparables, usando una restricción genérica.

Paso 1 — Definir la restricción (interfaz).

```
interface Comparable<T> {
  compareTo(other: T): number
}
```

`Comparable<T>` declara una operación `compareTo` que devuelve un número: negativo si `this < other`, cero si son iguales, positivo si `this > other`.

Paso 2 — Implementar `sortedSet` con la restricción.

```
class SortedSet<T extends Comparable<T>> {
  #items: T[] = []

  add(item: T): void {
    this.#items.push(item)
    this.#items.sort((a, b) => a.compareTo(b))
  }

  contains(item: T): boolean {
    return this.#items.some(x => x.compareTo(item) === 0)
  }

  toArray(): readonly T[] {
    return [...this.#items]
  }
}
```

La cláusula `T extends Comparable<T>` exige que el tipo `T` implemente `Comparable<T>`. Si intentás crear un `SortedSet<number>`, el compilador verifica que `number` implemente `compareTo`. (En TypeScript, los primitivos no implementan `Comparable` directamente; en la práctica se usa un `Comparator` externo o un wrapper. El patrón conceptual es el que importa.)

Paso 3 — Usar el conjunto con un tipo comparable.

```
class Persona implements Comparable<Persona> {
    constructor(public readonly nombre: string) {}

    compareTo(other: Persona): number {
        return this.nombre.localeCompare(other.nombre)
    }
}

const conjunto = new SortedSet<Persona>()
conjunto.add(new Persona("Zoe"))
conjunto.add(new Persona("Ana"))
conjunto.add(new Persona("Mario"))
console.log(conjunto.toArray().map(p => p.nombre))
// ["Ana", "Mario", "Zoe"] - ordenado alfabéticamente
```

Análisis: La restricción `T extends Comparable<T>` es **recursiva**: el tipo `T` se refiere a sí mismo en la restricción. Esto es común en patrones como `Comparable`, `Cloneable` o `Iterable`. El genérico no es solo un placeholder: es una restricción que el compilador hace cumplir. Esto es **polimorfismo paramétrico acotado** (bounded parametric polymorphism), distinto del polimorfismo paramétrico sin restricciones de `Stack<T>`.

6. Puntos clave — Cheat sheet

Concepto	Definición en una línea
TAD	Tipo definido por nombre + operaciones + semántica observable + invariantes; la representación queda oculta.
Encapsulamiento	Agrupar datos y operaciones en una unidad.
Ocultamiento de información	Proteger decisiones internas que pueden cambiar.
Interfaz pública	Promesas que recibe el cliente (contrato).
Invariante	Propiedad que debe ser siempre cierta (ej: LIFO).
Contrato de interfaz	Lo que publico es lo que asumo; publicar de más = asumir promesas de más.
Copia defensiva	Devolver copia o vista de solo lectura para evitar aliasing de la representación.
Independencia de representación	Poder cambiar el <i>cómo</i> sin romper el <i>quién</i> depende del contrato.
Módulo como frontera	El módulo decide qué nombres son públicos (<code>export</code>) y cuáles internos.
Genéricos	<code>Stack<T></code> parametriza el tipo una sola vez: polimorfismo paramétrico.
Constraint genérica	<code>T extends Comparable<T></code> exige que el tipo cumpla una interfaz.
# vs private	<code>#</code> es privacidad de runtime (ES2022); <code>private</code> es solo de tipado.
import type	Dependencia solo del contrato (se borra en runtime).

Las cuatro ideas para llevarse:

1. Un **TAD** protege una representación.
2. Una **interfaz** transforma decisiones en promesas.
3. Un **módulo** controla qué nombres son públicos.
4. Un **genérico** reutiliza la abstracción sin mezclar tipos.

7. Autoevaluación

Diez preguntas organizadas por nivel Bloom. Las respuestas están colapsadas: intentá resolverlas primero y después abrí.

Pregunta 1 (Recordar)

¿Cuáles son los cuatro componentes que definen un TAD?

► Respuesta

Pregunta 2 (Recordar)

¿Qué símbolo usa TypeScript para privacidad real de runtime y en qué se diferencia de `private`?

► Respuesta

Pregunta 3 (Comprender)

¿Por qué LIFO es una invariante observable y el array no la garantiza?

► Respuesta

Pregunta 4 (Comprender)

Explicá la diferencia entre encapsulamiento y ocultamiento de información. ¿Qué error común produce confundirlos?

► Respuesta

Pregunta 5 (Aplicar)

Dado este método, ¿qué problema tiene y cómo lo corregís?

```
toArray(): T[] {  
  return this.#items  
}
```

► Respuesta

Pregunta 6 (Aplicar)

Clasificá las siguientes operaciones en constructoras, transformadoras u observadoras, y decidí cuál NO pertenece al TAD pila: `push`, `peek`, `isEmpty`, `at(index)`, `new Stack<T>()`.

► Respuesta

Pregunta 7 (Analizar)

¿Por qué `at(index)` rompe la abstracción de pila pero `clear()` no? Justificá usando el concepto de semántica observable.

► Respuesta

Pregunta 8 (Analizar)

Compará los cuatro tipos de import en TypeScript. ¿Cuál genera el acoplamiento más débil y por qué?

► Respuesta

Pregunta 9 (Evaluar)

Dado un contrato `Stack<T>` con `push` / `pop` / `peek` / `size`, un compañero propone agregar `toArray(): T[]` que devuelve el array interno “para que sea más fácil iterar”. ¿Es válido? Justificá usando los conceptos de la clase.

► Respuesta

Pregunta 10 (Crear)

Escribí una interfaz `IQueue<T>` (TAD cola, FIFO) con sus operaciones. Clasificá cada operación en constructora/transformadora/observadora. Indicá qué operación NO pertenecería al TAD cola y por qué.

► Respuesta

8. Glosario

Término	Definición
TAD (Tipo Abstracto de Datos)	Tipo definido por nombre, operaciones, semántica observable e invariantes. La representación interna queda oculta al cliente.
ADT (Abstract Data Type)	Equivalente en inglés de TAD.
Encapsulamiento	Agrupar datos y operaciones que forman una unidad conceptual en una sola entidad (clase, módulo).
Ocultamiento de información	Proteger las decisiones internas de implementación que pueden cambiar, para que los cambios no rompan a los clientes.
Interfaz	Especificación pública de las operaciones que un tipo ofrece. En TypeScript, se declara con <code>interface</code> . Es el contrato.
Módulo	Unidad de organización del código que declara una frontera de visibilidad. En TypeScript, cada archivo es un módulo. Decide qué nombres son públicos (<code>export</code>) y cuáles internos.
Export	Palabra clave que marca un nombre como parte de la frontera pública del módulo.
Import	Mecanismo para usar nombres definidos en otro módulo. Puede ser de tipos (<code>import type</code>), de implementación, funcional o por efecto lateral.
Genérico	Constructo que parametriza un tipo (<code>Stack<T></code>) para reutilizar la misma abstracción con tipos distintos sin duplicar código.
Constraint (restricción genérica)	Cláusula <code>T extends X</code> que exige que el tipo parámetro cumpla una interfaz. Ej: <code>T extends Comparable<T></code> .
Parametric polymorphism	Polimorfismo paramétrico: mismo cuerpo de código opera sobre tipos distintos mediante parámetros de tipo. Distinto del ad hoc polymorphism (sobrecarga).
Copia defensiva	Patrón que devuelve una copia o vista de solo lectura de la estructura interna para evitar que el cliente la mute directamente (aliasing).
Independencia de representación	Propiedad que permite cambiar la representación interna de un TAD sin romper a los clientes, porque estos dependen del contrato, no de la implementación.
Especificación	Declaración del contrato (qué promete el tipo). En TypeScript, la <code>interface</code> .
Implementación	Cuerpo que decide cómo se cumple el contrato. En TypeScript, la <code>class</code> que <code>implements</code> una interfaz.
Frontera de visibilidad	Límite que un módulo establece entre lo público (<code>export</code>) y lo interno. El módulo es la unidad de visibilidad, no la clase.

Término	Definición
Dependencia	Relación de un módulo con otro mediante imports. Puede ser de tipos, de implementación, funcional o por efecto lateral. Cada import es una decisión de acoplamiento.
Namespace	Mecanismo para agrupar nombres bajo un prefijo y evitar colisiones en el espacio global. C++ lo llama <code>namespace</code> ; TypeScript usa módulos (cada archivo aísla su espacio de nombres).

9. Referencias y lecturas recomendadas

Bibliografía citada (verificada en ChromaDB, `--type material`)

1. **Sebesta, R. W. — *Concepts of Programming Languages* (Pearson, 10ª ed., 2019), Cap. 11, pp. 471-506.** Abstract data types, encapsulation, information hiding, modules. Soporta los bloques 1, 2, 3, 4, 5, 6, 7, 8.
2. **Sebesta, R. W. — *Concepts of Programming Languages* (Pearson, 2019), Cap. 9, pp. 389-440.** Generic subprograms, parametric polymorphism, ad hoc polymorphism (sobrecarga). Soporta el bloque 11 (genéricos).
3. **Gabbrielli, M. & Martini, S. — *Programming Languages: Principles and Paradigms* (Springer, 2ª ed., 2023), Cap. 9, pp. 283-294.** ADTs y módulos; imports y visibilidad; clasificación de operaciones (constructoras, transformadoras, observadoras). Soporta los bloques 2, 9, 10.
4. **Gabbrielli, M. & Martini, S. — *Programming Languages: Principles and Paradigms* (Springer, 2023), Cap. 9, pp. 295-350.** Encapsulation and information hiding; módulos como partición estática con visibilidad. Soporta los bloques 3, 7.
5. **Gabbrielli, M. & Martini, S. — *Programming Languages: Principles and Paradigms* (Springer, 2023), Cap. 8, §Def. 8.6, p. 136.** Definición formal de polimorfismo paramétrico universal. Soporta el bloque 11.
6. **Louden, K. C. & Lambert, K. A. — *Programming Languages: Principles and Practices* (Course Technology, 2012), Cap. 11, pp. 496-545.** ADT mechanisms and modules; especificación algebraica de TADs; compilación separada; criterios de modificabilidad, reusabilidad y seguridad. Soporta los bloques 2, 8, 9, 10.

Lecturas recomendadas (profundización opcional)

- Sebesta, Cap. 11, §11.4 — Ejemplos de TADs en C++, Java, C# y Ruby. Útil para comparar cómo distintos lenguajes materializan el mismo concepto.
- Gabbrielli & Martini, Cap. 9, §10.1 — Limitaciones de los TADs aislados y motivación de los módulos como constructo que los agrupa.
- Louden & Lambert, Cap. 11, §11.3 — Compilación separada en C, C++ namespaces y Java packages. Útil para entender la frontera de visibilidad más allá de TypeScript.

Trazabilidad

- Las citas bibliográficas de esta guía fueron consultadas en ChromaDB (`scripts/knowledge_base.py search ... --type material`) y coinciden con la trazabilidad registrada en `minuta.md` (sección *Trazabilidad bibliográfica*).
 - El contenido teórico y los ejemplos de código son fieles al `clase_dada.txt` (baseline de la clase dada, 324 líneas).
 - El scope respeta `diseño.md` (objetivos 1-10, bloques TAD → genéricos).
-

 **Cierre de Sofía:** Si llegaste hasta acá y pudiste responder las diez preguntas sin mirar las respuestas, te llevás la clase. La próxima clase vamos del módulo aislado a la **composición de abstracciones**: cómo se combinan módulos sin perder las fronteras que hoy construimos. Si te trabaste en alguna pregunta, volvé al bloque correspondiente y releé el código del `clase_dada.txt` : ahí está la respuesta.